Lecture 3

29 July 2026 10:41

We focus on algorithmic problem, where the aim is convert specified input instance to specified output instance.

Consider Sorting problem:

Input: a set of n numbers $(a_1, a_2,...a_n)$ — Specification of input

Output: a set of n numbers $(b_1, b_2,...b_n)$ such that
- $b_1 <= b_2 <=...b_n$ and — sorted order
- $(b_1, b_2,...b_n)$ is a permutation of $(a_1, a_2,...a_n)$

} specification of output

There are infinite input instance possible.

The same sets of inputs can be given to any algorithm that can solve the problem.

No matter what sorting algorithm we use, the set of possible inputs always remains the same.

What about Case:

- A case is a specific input instance given to the algorithm
- A given case can be better or worse for a given algorithm.
- Some algorithms always perform the same no matter what the inputs are, but some algorithms might do worse on some inputs.
- Every algorithm may have some best case and some worst case.

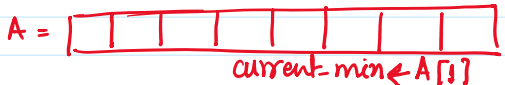
Algorithm Case Example:

- Finding minimum of an unsorted list:

Prob

Best case and worst case are same for this Algo

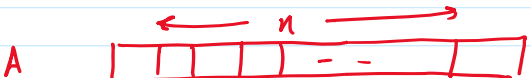
Input array of size n



Algo

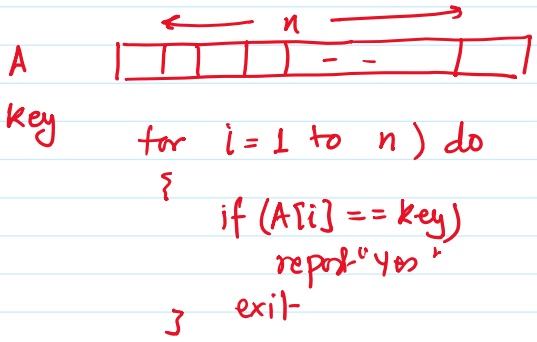
```
for ( i = 1 to n ) do
{
    current_min > A[i]
        current_min ← A[i]
}
```

- Finding a given element in a given list:



- Finding a given element in a given list:

Best case ≠ Worst case



Determining efficiency of algorithm as function of input size:

Determine running time as a function of input size $f(n)$

- Running time is computed by counting the number of primitive operations used in the algorithm $= f(n)$

- Two interesting cases: best case and worst case

- The best case is the input that minimizes the function, and the worst case is the input that maximizes the function.

$f(n)$ — counting # of primitive operation

$g(n)$ — nice / interpretable fn

- Best case and worst case are very concrete things w.r.t. a given algorithm.

Why do we need bounds:

Consider that running time: $n^2 + (\log n)! + e^{\frac{\log n}{n}}$

- For interpretability of running time function, we are interested in few specific type of functions
  Like constant, logarithmic, linear, quadratic, cubic, exponential

- An upper bound is a function that sits on top of another function

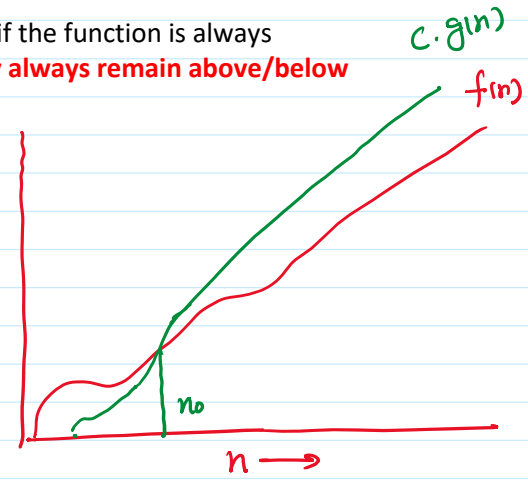
- A lower bound is a function that sits on below of another function

$f(n) \leq c \cdot g(n) \quad n \geq n_0$

- When we talk about Big Oh and Big Omega notation, we don't care if the function is always above/below the other function. Just that **after a certain point they always remain above/below other function.**

$\exists$ const c, and n_0 s.t.

$f(n) \leq c \cdot g(n) \quad \forall n \geq n_0$ Running time



Upper bound to be smallest upper bound

lower Bound to be largest lower Bound

- There are infinite function possible: obvious upper bound it infinity and lower bound is zero.

- Least upper bound and greatest lower bound are the most informative.

$\exists c, n_0$ s.t

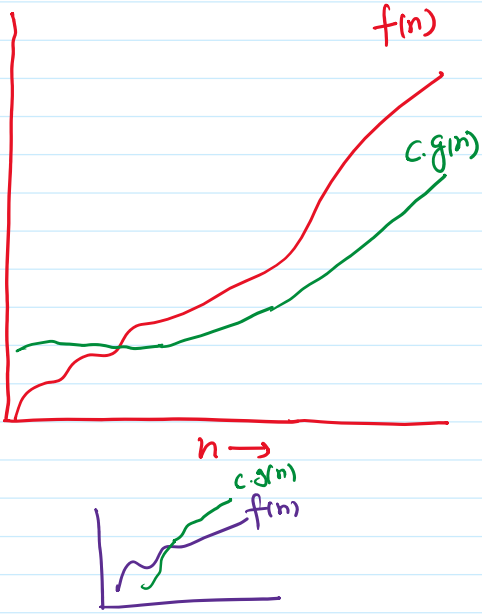
$f(n) \geq c \cdot g(n) \quad \forall n \geq n_0$

$T'(n)$

$O(A(n))$

$T(n)$: worst case running of Alg A

$T(n) \in O(n^2)$



Best/Worst Case + Lower/Upper Bound

Worst Case Lower Bound: A function that is a **boundary below the algorithms' runtime function**, when that algorithm is given the **input that maximize the running time**.

Worst Case Upper Bound: A function that is a **boundary above the algorithms' runtime function**, when that algorithm is given the **input that maximize the algorithm's run time**.

Best Case Lower Bound: A function that is a **boundary below the algorithms' runtime function**, when that algorithm is given the **inputs that minimize the algorithm's run time**.

Best Case Upper Bound: A function that is a **boundary above the algorithms' runtime function**, when that algorithm is given the **inputs that minimize the algorithm's run time**.

Examples:

Worst Case Lower Bound:

- **Comparison based sorting algorithm**: worst case lower bound is $\Omega(n \log n)$.

Best Case Lower Bound:

- **Insertion sort:** If the input is sorted, we only require to walk through the list without doing any insertions. Best case lower bound is $\Omega(n)$.

Worst Case Upper Bound:

- **Insertion sort:** If the input is sorted, in reverse order. Worst case upper bound is $O(n^2)$.

Best Case Upper Bound:

- **Finding an element in the list:** The best case is when the element we are looking for is the first element. The upper bound is $O(1)$.

While designing algorithms we mostly care about worst case lower/upper bound.

Worst case input instance of one algo can be best case input instance of another algo and vice versa.

Sorting Problem

Insertion Sort
- Best case : when input is already sorted
- Worst case : when input is sorted in reverse order

Quick Sort
- Best case : When input is sorted in reverse order
- Worst Case : when input is sorted